

บทที่ 1

บทนำ

1.1 ความสำคัญและที่มา

การพัฒนาซอฟต์แวร์เชิงคอมโพเนนต์ (Component-Based Software Development หรือ CBSD) เป็นกระบวนการในการสร้างซอฟต์แวร์ โดยนำคอมโพเนนต์ที่มีอยู่มาใช้ (reuse) เพื่อลดเวลาและค่าใช้จ่ายในการพัฒนา

และการพัฒนาสถาปัตยกรรมคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์ (Server-side component architecture) ยังช่วยให้การพัฒนาแอปพลิเคชันระดับเอ็นเตอร์ไพรส์ (Enterprise application) ทำได้ง่าย นอกจากนั้นยังมีความน่าเชื่อถือ (Reliable) , มีระบบรักษาความปลอดภัย (Security) ที่ถูกต้องมากขึ้น ทั้งยังช่วยลดความซับซ้อนเนื่องจากโครงสร้างของระบบ ในปัจจุบันได้มีบริษัทต่างๆ ได้พัฒนาสถาปัตยกรรมที่ทำงานบนฝั่งเซิร์ฟเวอร์ (Server-side architecture) 2 สถาปัตยกรรมที่ได้รับการยอมรับมากที่สุดคือ

- Microsoft's Distributed interNet Applications Architecture (DNA) เป็นสถาปัตยกรรมของบริษัทไมโครซอฟท์ (Microsoft) ที่นำเสนอสถาปัตยกรรมคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์คือ COM+ (Component Object Model plus)
- Sun Microsystem's Java 2 Platform, Enterprise Edition (J2EE) เป็นสถาปัตยกรรมของบริษัทซัน ไมโครซิสเต็มส์ (Sun Microsystems) ที่นำเสนอสถาปัตยกรรมคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์คือ EJB (Enterprise JavaBeans)

สถาปัตยกรรมของทั้ง 2 บริษัทมีความแตกต่างที่น่าสนใจคือ J2EE นำเสนอความสามารถในการทำงานข้ามแพลตฟอร์ม (portability) และ DNA นำเสนอความสามารถในการทำงานข้ามภาษา (interoperability) เนื่องจากทางบริษัทซันไมโครซิสเต็มส์ได้ออกแบบภาษาจาวา (Java) ให้สามารถทำงานข้ามแพลตฟอร์ม การพัฒนาคอมโพเนนต์ด้วยภาษาจาวาทำให้คอมโพเนนต์มีความสามารถนี้อยู่ด้วย ในขณะที่ทางบริษัทไมโครซอฟท์ได้สนับสนุนการสร้างคอมโพเนนต์จากภาษาใดก็ได้เช่น Visual Basic และ Visual C++ คอมโพเนนต์ที่สร้างด้วย Visual Basic สามารถนำมาใช้กับ Visual C++ ได้ ความแตกต่างอีกอย่างหนึ่งคือ J2EE เป็น specification เท่านั้น แอปพลิเคชันเซิร์ฟเวอร์ (Application Server) จะถูกพัฒนาโดยผู้ผลิตรายอื่น เช่น BEA WebLogic Server , IBM WebSphere , Oracle 9i Application Server , SilverStream Application Server , iPlanet , Sybase Enterprise Application Server , Borland AppServer เป็นต้น

ทั้ง 2 สถาปัตยกรรมนี้มีข้อจำกัดที่ไม่สามารถนำกลับมาใช้ใหม่ (reuse) ข้ามสถาปัตยกรรมได้ แต่ภายหลังการเกิดของ XML , SOAP และแนวคิดของเว็บเซอร์วิส (Web Service) ทำให้สามารถทำงานร่วมกันได้ผ่านทาง SOAP ซึ่งเป็นโพรโตคอลที่ใช้สำหรับ Remote Procedure Call (RPC) ผ่าน HTTP โพรโตคอล

1.2 วัตถุประสงค์ของโครงการ

- 1.2.1 ศึกษาการออกแบบและพัฒนาซอฟต์แวร์เชิงคอมโพเนนต์
- 1.2.2 ศึกษาสถาปัตยกรรมที่ทำงานบนฝั่งเซิร์ฟเวอร์ ข้อดีและข้อเสียของแต่ละสถาปัตยกรรม
- 1.2.3 ศึกษา XML และ SOAP
- 1.2.4 สร้างแอปพลิเคชันโดยใช้สถาปัตยกรรมคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์ และสามารถนำคอมโพเนนต์มาใช้ใหม่ข้ามสถาปัตยกรรมได้
- 1.2.5 ศึกษาและพัฒนาเว็บเซอร์วิส

1.3 ขอบเขตของโครงการ

โครงการนี้พัฒนาระบบอี-คอมเมิร์ซที่เกี่ยวกับการรับจัดงานแต่งงานโดยใช้ EJB พัฒนาคอมโพเนนต์, และเว็บเซอร์วิสด้วยเว็บโลจิกแพลตฟอร์ม 7.0 แสดงผลด้วย JSP ที่เรียกใช้คอมโพเนนต์ผ่านทางเว็บเซอร์วิส ใช้ SOAP (Simple Object Access Protocol) ในการทำหน้าที่เป็นสื่อกลางในเรียกใช้เว็บเซอร์วิสของไมโครซอฟท์ (DOT NET FRAMEWORK)

1.4 วิธีการดำเนินงาน

การศึกษาในโครงการเริ่มจากศึกษาทฤษฎีพื้นฐานของการพัฒนาเชิงคอมโพเนนต์และเว็บเซอร์วิส , J2EE ซึ่งเป็นเทคโนโลยีสถาปัตยกรรมบนฝั่งเซิร์ฟเวอร์ที่ใช้ภาษาจาวาในการพัฒนา ประกอบด้วย EJB , JSP , JNDI และ JDBC เป็นต้น โดยใช้แนวคิดของการทำซอฟต์แวร์ให้มีลักษณะที่นำกลับมาใช้ใหม่ได้ และขยายเพิ่มเติมได้ง่าย และยังมีส่วนที่ใช้ในการเรียกใช้เว็บเซอร์วิส ข้ามสถาปัตยกรรม ประกอบด้วย XML (eXtensible Markup Language) และ SOAP (Simple Object Access Protocol) โดยจะแบ่งการดำเนินงานเป็น 2 ส่วนคือ ส่วนแอปพลิเคชัน จะศึกษาและออกแบบระบบอี-คอมเมิร์ซ ส่วนสถาปัตยกรรม (Architecture) เป็นส่วนของการออกแบบการทำ SSL (Security Socket Layer) เพื่อเพิ่มความปลอดภัยในการใช้งาน